# Design and Implementation of a 5-Stage Pipelined RISC-V Processor (RV32I)

Tools: Verilog, Icarus Verilog, GTKWave, Intel Quartus Prime

## 1. Introduction

The **RISC-V** instruction set architecture (ISA) is an open-standard architecture designed for flexibility, scalability, and efficient processor design. It follows the principles of Reduced Instruction Set Computing (RISC), enabling simplified hardware implementation and improved performance.

In this project, a **32-bit RISC-V processor based on the RV32I instruction set** is designed and implemented using Verilog HDL. The processor adopts a **5-stage pipelined architecture**, consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. Pipelining enhances performance by allowing multiple instructions to be processed simultaneously across different stages.

To ensure correct execution, a **hazard handling unit** is implemented to manage data and control hazards using forwarding, stalling, and pipeline flushing techniques. The processor supports essential instruction types including arithmetic, logical, memory, and branch operations.

The design is verified through simulation using tools such as Icarus Verilog and GTKWave, and further validated by executing programs assembly programs derived from C logic. Additionally, the processor is synthesized using Intel Quartus Prime, confirming its hardware feasibility.

This project demonstrates a complete processor design flow, including datapath design, control logic implementation, pipelining, hazard handling, simulation, and synthesis.

## 2. Project Objectives

The primary objectives of this project are:

- To design and implement a 32-bit RISC-V processor using Verilog HDL
- To develop a 5-stage pipelined architecture for efficient instruction execution
- To implement the RV32I base instruction set including arithmetic, logical, memory, and control instructions
- To design and integrate a hazard handling unit for resolving data and control hazards
- To enable execution of programs assembly programs derived from C logic
- To verify processor functionality using simulation tools such as Icarus Verilog and GTKWave
- To synthesize the design using Intel Quartus Prime and validate hardware feasibility
- To analyze pipeline performance and ensure correct instruction flow across all stages

## 3. Architecture

Pipeline: IF, ID, EX, MEM, WB

The designed processor follows a **5-stage pipelined architecture** based on the RV32I instruction set. Pipelining improves performance by allowing multiple instructions to be executed simultaneously across different stages.

**Pipeline Stages:**

- **Instruction Fetch (IF):**
The Program Counter (PC) generates the address of the next instruction, which is fetched from instruction memory.

- **Instruction Decode (ID):**
The fetched instruction is decoded, control signals are generated, and the required operands are read from the register file.

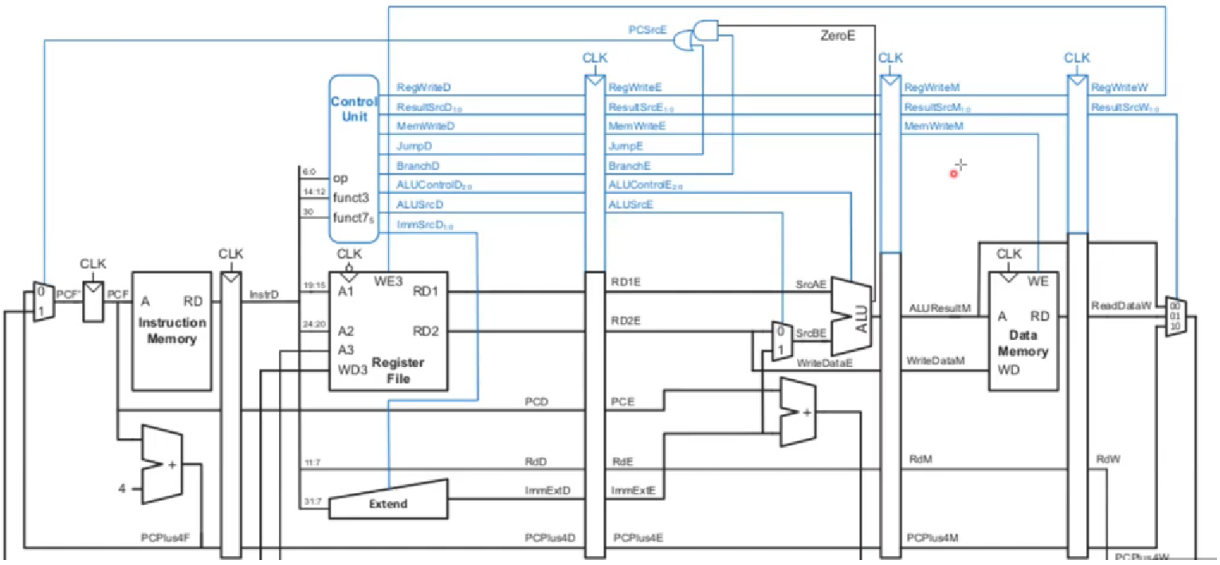
- **Execute (EX):**
The Arithmetic Logic Unit (ALU) performs arithmetic and logical operations. Branch conditions are also evaluated in this stage.

- **Memory Access (MEM):**
Load and store instructions access data memory for reading or writing data.

- **Write Back (WB):**
The final result is written back to the destination register in the register file.

The processor includes key components such as the Program Counter, Instruction Memory, Register File, ALU, Control Unit, and Data Memory. Pipeline registers are used between stages to maintain proper data flow and synchronization.

# Pipeline Datapath




## 4. Hazard Handling

In a pipelined processor, hazards may arise due to overlapping instruction execution. To ensure correct operation, a hazard handling unit is implemented.

**Data Hazards:**

Data hazards occur when an instruction depends on the result of a previous instruction. These are resolved using **forwarding (bypassing)**, where data is directly passed from later stages to earlier stages without waiting for write-back.

**Load-Use Hazards:**

When a load instruction is immediately followed by a dependent instruction, the data may not be available in time. This is handled using **stall logic**, which temporarily pauses the pipeline to ensure correct data availability.

**Control Hazards:**

Control hazards occur due to branch instructions. These are managed using **pipeline flushing**, where incorrect instructions in the pipeline are discarded when a branch decision is made.

The hazard handling mechanisms ensure correct execution while maintaining pipeline efficiency.

## 5. Execution Flow

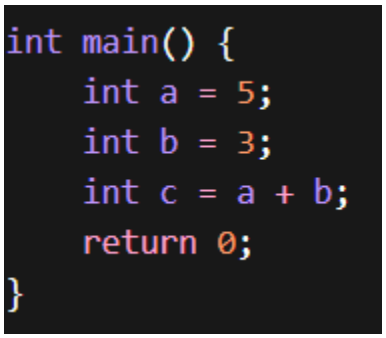
The processor supports execution of both manually written machine instructions and assembly programs derived from C logic.

**Execution Process:**

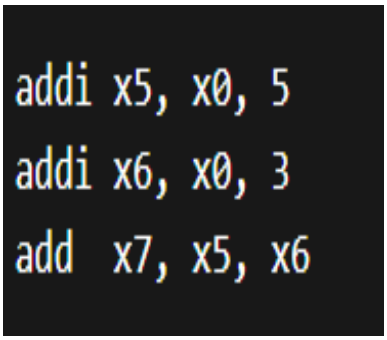
- A C program is written and converted using the Venus simulator
- The compiled output is converted into machine code (hex format)
- The machine code is stored in the instruction memory file (program.mem)
- During simulation, the processor fetches and executes instructions sequentially
- Results are written back to registers and verified through simulation outputs

### Execution Flow Diagram:

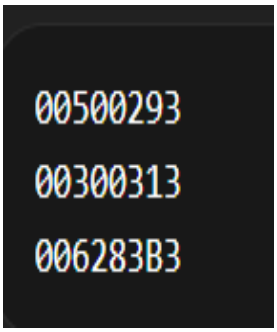
C Logic → Assembly → Venus (Assembler) → Machine Code → program.mem → CPU



```c
int main() {
    int a = 5;
    int b = 3;
    int c = a + b;
    return 0;
}
```

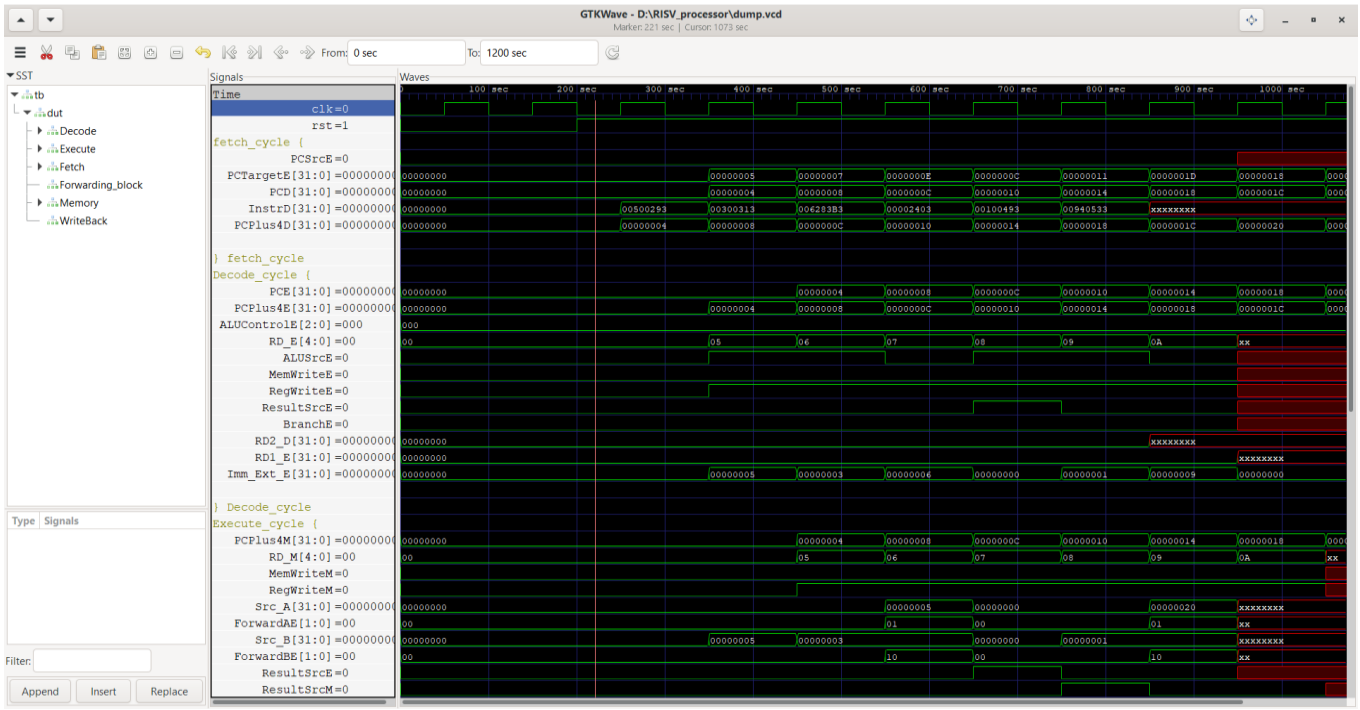


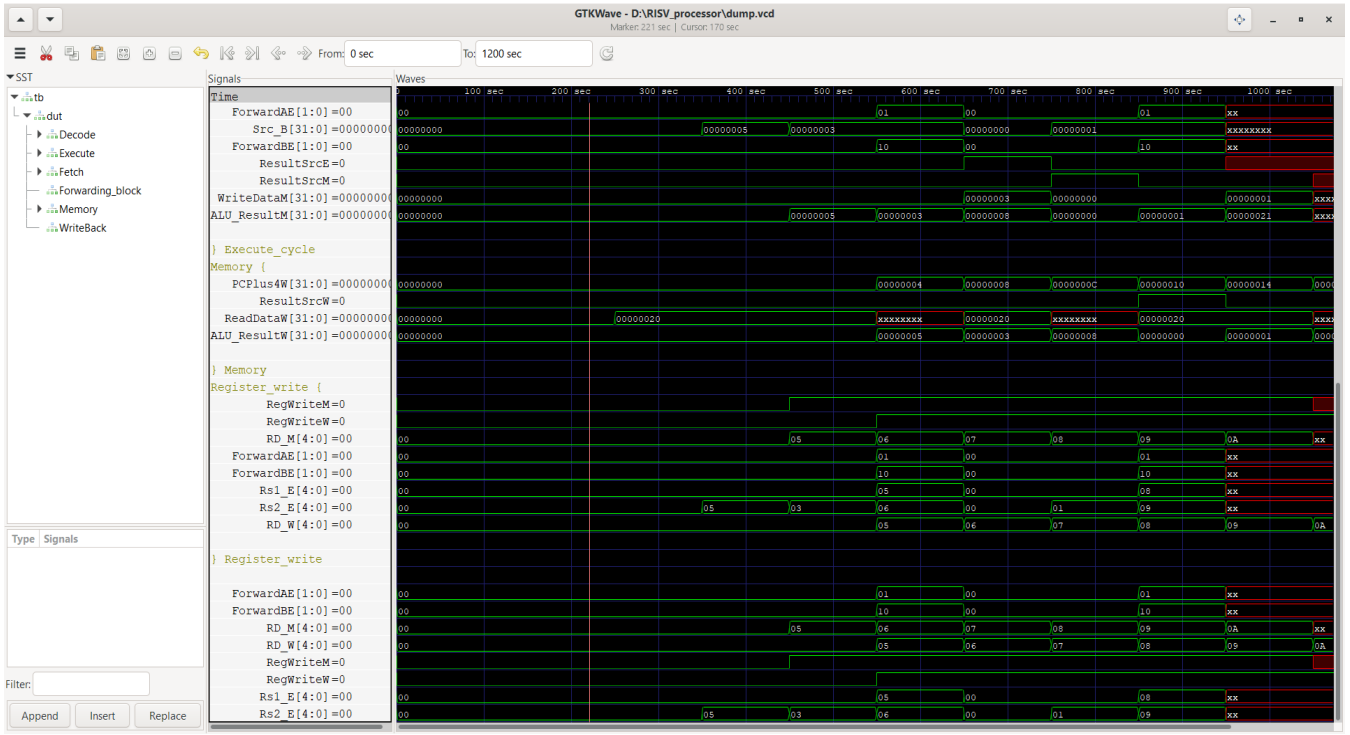
```
addi x5, x0, 5
addi x6, x0, 3
add  x7, x5, x6
```



```
00500293
00300313
006283B3
```

# 6. Simulation



## 7. Results

The designed 5-stage pipelined RISC-V processor was successfully verified through simulation. The processor executed instructions correctly across all pipeline stages, demonstrating proper functionality of the datapath, control logic, and hazard handling mechanisms.

**Functional Verification:**

- Correct instruction fetch and Program Counter (PC) progression observed
- Proper decoding and execution of arithmetic, logical, memory, and branch instructions
- Accurate register write-back in the Write Back stage
- Successful handling of data and control hazards using forwarding, stalling, and flushing

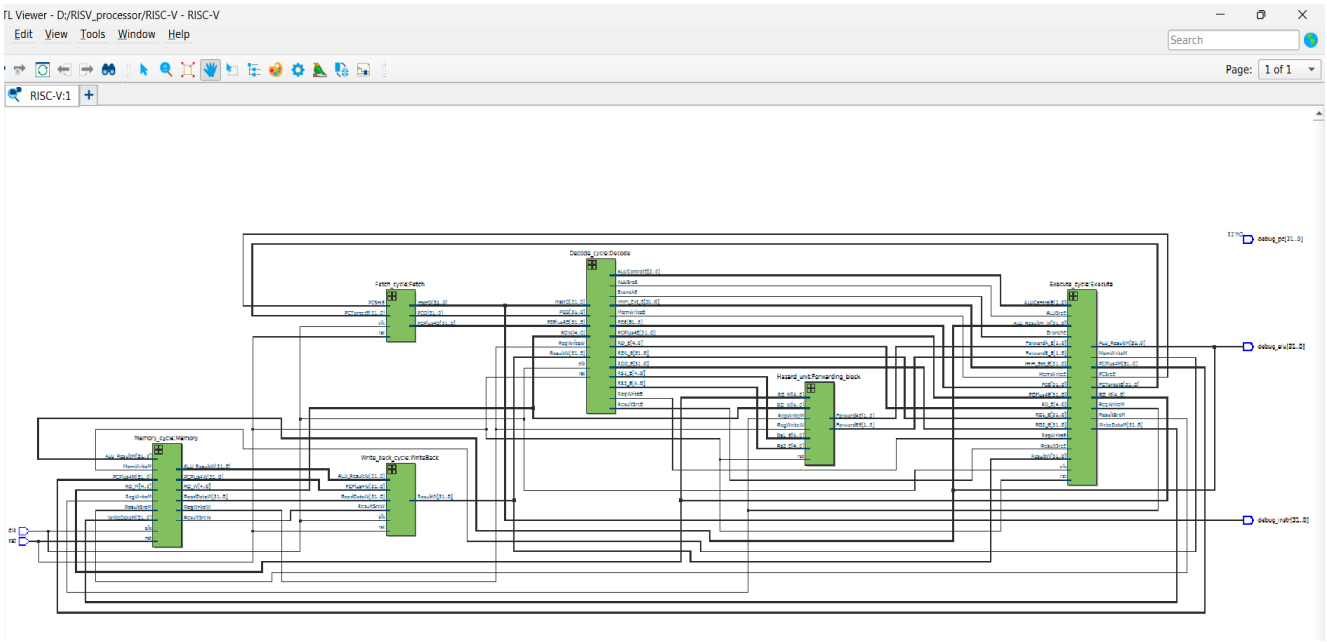
**Output Validation:**

The correctness of the processor was validated by observing the final register values after program execution:

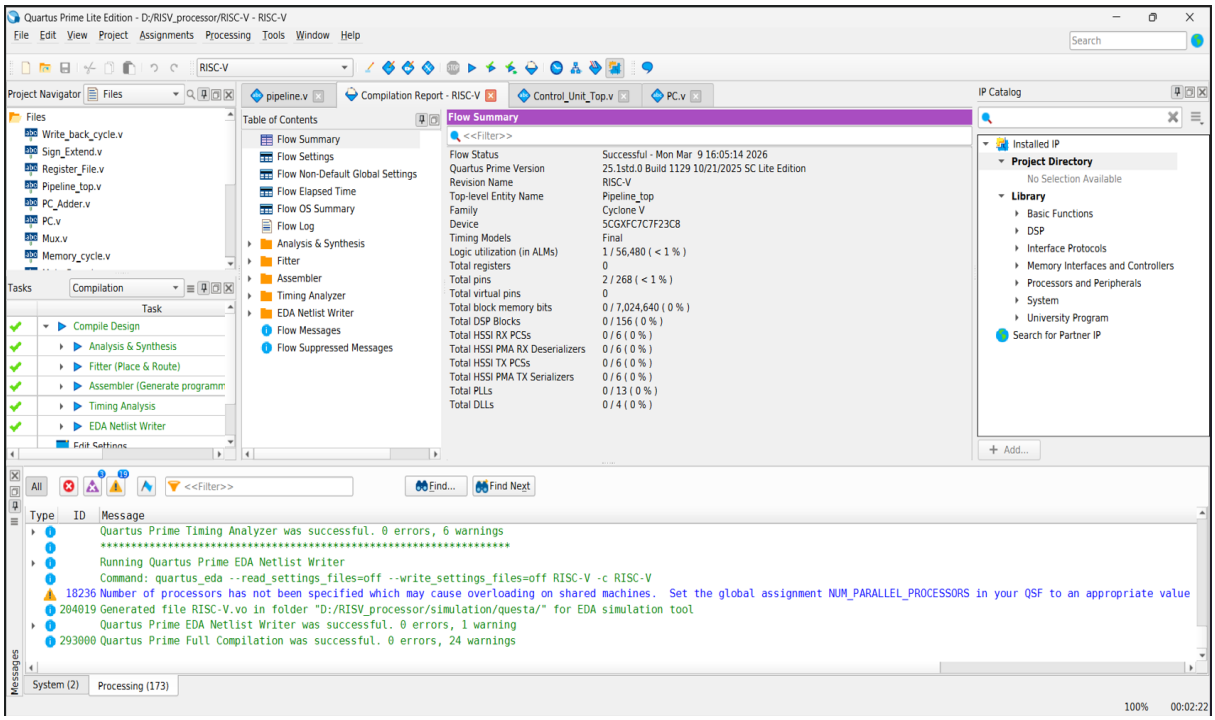
```
Final Registers:
x5 = 5
x6 = 3
x7 = 8
x8 = 32
x9 = 1
x10 = 33
```

**Simulation Analysis:**

- Pipeline stages operated concurrently without functional errors
- Hazard handling ensured correct data flow between dependent instructions
- Undefined values appeared only after program completion due to instruction memory limits, which is expected behavior




## 8. Quartus




**Synthesis Result:**

The design was successfully synthesized using Intel Quartus Prime, confirming that the processor is hardware feasible and suitable for FPGA implementation.

## 9. Conclusion

The design and implementation of a **32-bit 5-stage pipelined RISC-V processor (RV32I)** has been successfully completed using Verilog HDL. The processor demonstrates correct functionality across all pipeline stages, including instruction fetch, decode, execute, memory access, and write-back.

The integration of a **hazard handling unit** ensures proper execution in the presence of data and control dependencies, using forwarding, stalling, and pipeline flushing techniques. Simulation results confirm accurate instruction execution, correct register updates, and efficient pipeline operation.

The processor is capable of executing programs assembly programs derived from C logic, validating its functionality at the instruction set level. Additionally, successful synthesis using Intel Quartus Prime confirms the hardware feasibility of the design.

Overall, this project provides a comprehensive understanding of processor architecture, pipelining, hazard handling, and hardware verification, demonstrating a complete end-to-end processor design flow.